

# Relatório Técnico --- ACADeO

## Agente Comparador de Apólices D&O

### 1. Resumo executivo

O **ACADeO --- Agente Comparador de Apólices D&O** é um MVP educacional de aplicação de Inteligência Artificial Generativa voltado à extração, organização e comparação de informações presentes em apólices de seguro D&O.

A solução recebe documentos em PDF ou imagem, utiliza o Gemini para análise multimodal, organiza as informações em estruturas tipadas, armazena o resultado em SQLite e permite consultar e comparar duas apólices por meio de uma interface Streamlit.

A versão final utiliza uma arquitetura **multiagente**, composta por três etapas especializadas:

1. **Agente 1 --- Triagem**
2. **Agente 2 --- Extração Estrutural**
3. **Agente 3 --- Análise de Cláusulas**

A comparação entre apólices permanece determinística, implementada em Python, evitando que o modelo generativo seja responsável por concluir automaticamente qual contrato é juridicamente superior.

O projeto prioriza uma arquitetura simples, demonstrável e evolutiva, adequada ao escopo educacional.

### 2. Problema

Apólices de seguro D&O são documentos extensos e não estruturados, com informações distribuídas entre quadro-resumo, condições gerais, cláusulas específicas e outras seções.

A comparação manual exige localizar informações, organizar campos equivalentes, verificar valores e identificar diferenças entre cláusulas e exclusões.

O ACADeO automatiza principalmente o trabalho mecânico de localização e organização dessas informações, mantendo a análise humana para situações em que diferenças textuais ou contratuais exigem interpretação especializada.

### 3. Objetivo

#### Objetivo Geral

Desenvolver um MVP capaz de demonstrar, de forma integrada, como técnicas de Inteligência Artificial Generativa podem ser aplicadas à análise de documentos de seguro D&O.

A proposta não se limita à extração de informações isoladas de uma apólice, mas busca construir um fluxo completo no qual um documento não estruturado é transformado em dados organizados, armazenados e, posteriormente, utilizados na comparação entre diferentes apólices.

#### Fluxo Proposto

1. **Entrada** O usuário envia uma apólice em formato PDF ou imagem.

## Relatório Técnico - ACADeO

**2. Triagem** O sistema verifica a natureza do material recebido e identifica se ele é compatível com o domínio de seguro D&O, evitando que documentos incompatíveis sejam tratados como apólices válidas nas etapas seguintes.

**3. Extração** O sistema identifica informações relevantes como dados da seguradora, vigência, valores, coberturas, condições contratuais e exclusões, organizando tudo em uma estrutura padronizada que permite analisar documentos diferentes sob uma mesma representação de dados.

**4. Validação e Armazenamento** Os resultados da extração são validados e armazenados localmente, permitindo consultar uma apólice já analisada sem reprocessar toda a análise generativa e viabilizando a seleção de duas apólices previamente processadas para comparação.

**5. Comparação** O sistema verifica valores numéricos quando disponíveis, identifica informações textuais iguais ou diferentes e apresenta diferenças entre listas de exclusões. O ACADeO não transforma diferenças em decisão jurídica automática: quando duas redações são diferentes, sinaliza a diferença e indica que ela requer análise, preservando a interpretação humana.

**6. Observabilidade** A execução dos três agentes — Triagem, Extração Estrutural e Análise de Cláusulas — é apresentada na interface Streamlit e também no terminal, demonstrando visualmente a arquitetura multiagente utilizada.

### Síntese

Construir um fluxo capaz de receber, classificar, extrair, estruturar, validar, armazenar, consultar e comparar informações de apólices D&O, utilizando IA Generativa nas etapas de interpretação documental e regras determinísticas nas etapas em que a previsibilidade da comparação é mais importante.

## 4. Escopo

O MVP contempla o fluxo necessário para demonstrar a análise e comparação de apólices, desde o recebimento do documento até a apresentação dos resultados. A solução utiliza três agentes especializados, valida os dados extraídos, mantém as análises em SQLite e disponibiliza consulta e comparação pela interface Streamlit.

O MVP contempla:

- upload de PDF e imagens;
- análise multimodal com Gemini;
- três agentes especializados;
- respostas em JSON estruturado;
- validação com Pydantic;
- persistência em SQLite;
- consulta das apólices processadas;
- detecção e reprocessamento de documentos já existentes;
- comparação de duas apólices;
- comparação numérica e textual;
- identificação de exclusões presentes somente em cada apólice;
- indicador heurístico de confiabilidade;
- alertas;
- interface Streamlit;
- exibição do andamento dos agentes no Streamlit e no terminal.

O projeto não pretende ser sistema comercial de subscrição, regulação ou aconselhamento jurídico.

## 5. Arquitetura

```
PDF / imagem
  ↓
Streamlit / app.py
  ↓
src/pipeline_multiagent.py
  ↓
Agente 1 – Triagem
  ↓
Agente 2 – Extração Estrutural
  ↓
Agente 3 – Análise de Cláusulas
  ↓
Consolidação / Pydantic
  ↓
SQLite / apolices.db
  ↓
Consulta / Comparação
```

O `app_zero.py` é a versão anterior da interface e não participa do fluxo final.

## 6. Tecnologias

- **Python:** linguagem principal.
- **Streamlit:** interface.
- **Google Gemini / Google GenAI:** análise multimodal e extração.
- **Pydantic:** estrutura e validação dos dados.
- **SQLite:** persistência local.
- **python-dotenv:** carregamento da credencial da API.

## 7. Uso de IA

A IA Generativa participa do projeto em duas dimensões.

**Como ferramenta de desenvolvimento:** apoio à elaboração e revisão de código, depuração, discussão arquitetural e documentação. Os resultados foram revisados e testados antes da incorporação.

**Como componente funcional:** o Gemini analisa os documentos e executa as três etapas especializadas. As respostas são solicitadas em JSON estruturado e consolidadas e validadas pelo Pydantic.

A IA não substitui a revisão humana das cláusulas.

## 8. Pipeline multiagente

**Agente 1 --- Triagem**

## Relatório Técnico - ACADeO

Classifica o documento, verifica se é relacionado a D&O e identifica seu tipo. Pode interromper a extração detalhada quando o documento não é classificado como D&O.

### Agente 2 --- Extração Estrutural

Extraí dados cadastrais, financeiros e de vigência, incluindo seguradora, tomador, SUSEP, tipo, vigência, LMI, limite agregado, prêmio e evidências.

### Agente 3 --- Análise de Cláusulas

Extraí base de reclamações, POS/franquia, Coberturas A/B/C, custos de defesa, prazos, retroatividade, territorialidade, exclusões e evidências.

### Consolidação

Os resultados são reunidos em uma única estrutura `ApoliceD0`, incluindo as evidências produzidas pelos agentes.

## 9. Status dos agentes

A versão final utiliza um callback entre pipeline e interface. O pipeline envia os estados `processando` e `concluído` para cada agente.

Exemplo:

```
🔄 Agente 1 - Triagem: processando...
✅ Agente 1 - Triagem: concluído
🔄 Agente 2 - Extração Estrutural: processando...
✅ Agente 2 - Extração Estrutural: concluído
🔄 Agente 3 - Análise de Cláusulas: processando...
✅ Agente 3 - Análise de Cláusulas: concluído
```

O mesmo andamento pode ser acompanhado no terminal.

## 10. Ingestão e extração

`receber()` verifica existência, extensão, arquivo não vazio e limite de 20 MB. O pipeline aceita PDF, PNG, JPG, JPEG e WEBP.

Cada agente utiliza o SDK Google GenAI com documento multimodal, prompt especializado, resposta JSON estruturada e temperatura baixa. Há tratamento para indisponibilidade temporária e cota da API.

## 11. Dados e validação

`ApoliceD0` organiza seguradora, tomador, SUSEP, tipo, vigência, base de reclamações, valores, coberturas, condições, exclusões, evidências, confiabilidade e alertas.

`Evidencia` associa campo, página e trecho literal.

Após a consolidação, Pydantic valida a estrutura. O sistema calcula uma confiabilidade heurística ( `alta`, `media` ou `baixa` ) e registra alertas. Essa classificação não representa probabilidade estatística de acerto.

## 12. Persistência

O banco é:

```
vectorbase/apolices.db
```

A tabela `apolices` armazena `id`, arquivo, data de processamento, seguradora, tipo, confiabilidade, LMI e o JSON completo.

O JSON completo permite recuperar a análise sem nova chamada ao Gemini.

Apesar do nome `vectorbase`, não existe banco vetorial nesta versão. O diretório é uma área de persistência e ponto de extensão futuro.

### 13. Comparação

A função `comparar()` é determinística e executada em Python.

Para LMI e limite agregado, verifica ausência, igualdade e diferença percentual quando ambos os valores existem.

Para informações textuais, identifica ausência, igualdade ou diferença e sinaliza textos diferentes para análise humana.

As exclusões são comparadas por diferença literal entre as listas extraídas. Isso não equivale a interpretação jurídica semântica.

### 14. Interface

A interface final possui:

- **Upload e Análise:** upload, processamento, status dos agentes, reprocessamento, dados, confiabilidade e alertas;
- **Apólices Salvas:** consulta dos registros;
- **Comparar Apólices:** seleção de duas apólices e apresentação das diferenças.

### 15. Tratamento de erros

O pipeline possui tratamento para erros relacionados tanto ao recebimento dos documentos quanto à comunicação com a API Gemini e à consulta dos dados armazenados. Entre as situações tratadas estão:

- arquivo inexistente;
- extensão não suportada;
- arquivo vazio;
- arquivo acima do limite;
- ausência da chave Gemini;
- indisponibilidade temporária;
- cota esgotada;
- JSON incompatível;
- apólice inexistente.

Em caso de indisponibilidade temporária do Gemini, o pipeline realiza novas tentativas antes de interromper o processamento.

## 16. Decisões arquiteturais

**Multiagente:** separa responsabilidades em triagem, extração estrutural e análise de cláusulas.

**Comparação determinística:** mantém regras numéricas e literais reproduzíveis fora do modelo generativo.

**Gemini multimodal:** adequado ao processamento de documentos PDF/imagem.

**Pydantic:** valida a resposta estruturada.

**SQLite:** simples e adequado ao protótipo.

**Streamlit:** permite demonstração funcional com baixa complexidade.

**Sem RAG/vector database:** não são necessários para a comparação atual, baseada em campos estruturados.

## 17. Limitações

Como todo MVP baseado em Inteligência Artificial Generativa, o ACADeO possui limitações que precisam ser consideradas na interpretação de seus resultados. Essas limitações não representam necessariamente falhas do projeto, mas delimitam aquilo que pode ser afirmado a partir das informações produzidas pelo sistema.

A primeira limitação está relacionada à própria natureza dos modelos generativos. Os resultados da extração podem variar de acordo com o documento, sua organização e as características do conteúdo analisado. Mesmo utilizando prompts especializados, schemas estruturados e temperatura baixa, não é possível tratar o resultado de um modelo generativo como uma garantia absoluta de extração correta.

A qualidade do documento também exerce influência direta sobre o processamento. PDFs ou imagens com baixa legibilidade, organização incomum ou informações difíceis de localizar podem reduzir a capacidade de extração. Dessa forma, o sistema depende não apenas do modelo, mas também da qualidade e das características do material fornecido.

Existe ainda uma dependência operacional da API Gemini. A disponibilidade do serviço, limites de utilização e cotas podem interferir no processamento. O pipeline possui mecanismos de novas tentativas para situações temporárias, mas não elimina a dependência do serviço externo.

Outra limitação importante está na comparação textual. O fato de duas apólices apresentarem textos diferentes não significa, por si só, que uma ofereça uma cobertura superior ou inferior à outra. Uma alteração na redação pode decorrer de diferenças de estilo, estrutura ou terminologia. Por esse motivo, o ACADeO sinaliza a diferença e indica que ela requer análise, em vez de produzir uma conclusão jurídica.

As exclusões apresentam uma limitação semelhante. Na versão atual, elas são comparadas principalmente pela diferença entre os textos extraídos. Portanto, uma exclusão identificada somente em uma apólice não deve ser interpretada automaticamente como prova de que o risco correspondente está necessariamente coberto pela outra. A interpretação depende do conjunto das condições contratuais.

O indicador de confiabilidade também possui natureza heurística. As categorias **alta**, **media** e **baixa** não representam uma probabilidade estatística de acerto. Elas funcionam como um mecanismo auxiliar para indicar a quantidade e a qualidade estrutural das informações obtidas.

Por consequência, a revisão humana continua sendo necessária. O ACADeO foi desenvolvido para reduzir o trabalho mecânico de localização e organização das informações, e não para substituir a leitura profissional de uma apólice ou produzir uma decisão jurídica automática.

## Relatório Técnico - ACADeO

Do ponto de vista de infraestrutura, SQLite e execução local são adequados ao objetivo do MVP, mas não foram escolhidos para cenários de alta concorrência, processamento distribuído ou operação em larga escala. Da mesma forma, a versão atual não possui mecanismos de autenticação, autorização ou infraestrutura de produção.

O projeto também não possui integrações externas com seguradoras ou outros sistemas corporativos. O processamento ocorre a partir dos documentos fornecidos diretamente pelo usuário.

Por fim, apesar do diretório `vectorbase` existir na estrutura do projeto, não há um banco vetorial operacional nessa versão. O diretório atualmente funciona como área de persistência do SQLite e como ponto de extensão arquitetural para uma possível evolução envolvendo RAG ou busca semântica.

As principais limitações podem, portanto, ser resumidas em:

1. resultados generativos podem variar;
2. documentos pouco legíveis podem reduzir a qualidade da extração;
3. o processamento depende da disponibilidade e da cota da API Gemini;
4. comparação textual não significa diferença jurídica;
5. exclusões são comparadas literalmente;
6. o indicador de confiabilidade é heurístico;
7. revisão humana continua necessária;
8. SQLite e execução local não são escolhas destinadas à alta escala;
9. não há autenticação ou infraestrutura de produção;
10. não há integrações externas;
11. não há banco vetorial operacional.

Essas limitações são compatíveis com o caráter educacional e de prova de conceito do ACADeO e também ajudam a delimitar claramente as possibilidades de evolução da solução.

## 18. Evolução futura

A arquitetura atual foi concebida de forma que novas funcionalidades possam ser incorporadas sem alterar o princípio fundamental do MVP. A evolução mais natural está relacionada à ampliação da capacidade de recuperação e interpretação das informações extraídas.

Uma possibilidade é incorporar **RAG e banco vetorial**, permitindo que documentos e trechos relevantes sejam indexados e posteriormente recuperados por similaridade semântica. Isso poderia ser utilizado, por exemplo, para localizar cláusulas semelhantes entre diferentes apólices sem depender exclusivamente da correspondência literal dos textos.

A comparação também poderia evoluir de uma abordagem predominantemente textual para uma comparação semântica. Nesse cenário, o sistema poderia identificar que duas cláusulas tratam de um mesmo tema mesmo quando utilizam redações significativamente diferentes. Essa evolução, entretanto, exigiria mecanismos adicionais de validação para evitar que uma semelhança linguística fosse interpretada como equivalência jurídica.

além dessas, existem:

- classificação mais detalhada de cláusulas;
- exibição de evidências na interface;
- geração de relatório comparativo;
- API;

- banco externo;
- autenticação e controle de acesso;
- observabilidade;
- avaliação sistemática da qualidade.

## 19. Execução

```
python -m venv .venv  
source .venv/bin/activate  
pip install -r requirements.txt
```

Configure:

```
GOOGLE_API_KEY=sua_chave_aqui
```

Execute:

```
streamlit run app.py
```

## 20. Conclusão

O ACADeO demonstra a aplicação integrada de Inteligência Artificial Generativa ao processamento de documentos de seguro D&O. A solução parte de um documento não estruturado e conduz esse conteúdo por uma sequência de etapas especializadas, transformando-o em informações estruturadas que podem ser persistidas, consultadas e comparadas.

A arquitetura final utiliza três agentes com responsabilidades distintas. A Triagem realiza a classificação inicial do documento; a Extração Estrutural concentra-se nas informações cadastrais, financeiras e de vigência; e a Análise de Cláusulas trabalha com coberturas, condições, prazos, territorialidade e exclusões. Essa divisão torna o processamento mais organizado e permite apresentar ao usuário, inclusive durante a execução, quais etapas estão sendo realizadas.

Depois da atuação dos agentes, os resultados são consolidados e validados por meio do Pydantic. A estrutura validada é então armazenada no SQLite, permitindo que as informações sejam recuperadas posteriormente sem a necessidade de executar novamente o processamento generativo.

A comparação representa uma decisão importante da arquitetura. Embora a Inteligência Artificial seja utilizada para interpretar o conteúdo dos documentos, a etapa de comparação foi mantida em Python e baseada em regras determinísticas. Dessa maneira, operações como identificação de valores ausentes, igualdade de informações e cálculo de diferenças numéricas não dependem de uma nova geração do modelo.

O fluxo completo pode ser representado da seguinte forma:

## Relatório Técnico - ACADeO

```
graph TD; Documento --> Triagem; Triagem --> Extração_Estrutural[Extração Estrutural]; Extração_Estrutural --> Análise_de_Cláusulas[Análise de Cláusulas]; Análise_de_Cláusulas --> Pydantic; Pydantic --> SQLite; SQLite --> Comparação; Comparação --> Interface;
```

Documento  
↓  
Triagem  
↓  
Extração Estrutural  
↓  
Análise de Cláusulas  
↓  
Pydantic  
↓  
SQLite  
↓  
Comparação  
↓  
Interface

Um aspecto central do projeto é que a automação não é apresentada como substituição da análise profissional. Quando o sistema encontra textos diferentes, por exemplo, ele não afirma automaticamente que uma apólice é melhor ou pior. Em vez disso, apresenta a diferença e indica que ela requer análise. O mesmo princípio é aplicado às exclusões e às demais condições contratuais.

Dessa forma, o principal resultado do ACADeO não é apenas a utilização de um modelo generativo, mas a construção de um fluxo integrado no qual IA Generativa, estruturação de dados, validação, persistência e regras determinísticas trabalham em conjunto.

A arquitetura foi mantida deliberadamente simples para facilitar a compreensão do funcionamento, a demonstração do MVP, a avaliação acadêmica e a evolução futura. Ao mesmo tempo, a separação entre interface, agentes, estrutura de dados, persistência e comparação estabelece uma base adequada para futuras extensões, como RAG, busca semântica, geração de relatórios, APIs e mecanismos mais avançados de avaliação e rastreabilidade.